



Universidad Autónoma de Baja California

FACULTAD DE CIENCIAS QUÍMICAS E INGENIERÍA



Programación orientada a objetos.

Laboratorio No. 5: Asociación y Composición (Parte 2)

Alumno:

Joshua Osorio O. – 1293271

Docente:

Gustavo O. Zamarron De La Garza

Septiembre/2026

Tabla de contenido

I.	Objetivo	3
II.	Descripción de la practica.....	3
	Lista de materiales.....	3
III.	Desarrollo de la práctica	4
	Metodología y Arquitectura	4
	Descripción del problema y elementos clave	4
	Datos de entrada y datos de salida	4
	Diagrama de clases.....	5
	Fragmentos de código y visualización de elementos clave	6
	Clase Empresa	6
	Clase Empleado	6
	Clase Equipo.....	7
	Clase Main — menú principal	7
	Reflexión y análisis crítico	8
	Dificultades técnicas.....	8
	Fracasos, fallas, bugs y errores	8
	Cómo se resolvió.....	9
	Conclusión	9
IV.	Bibliografía.....	9

I. Objetivo

El alumno diseñará clases para modelar relaciones de asociación y composición entre entidades del mundo real, respetando los principios de la orientación a objetos y las capacidades del lenguaje Java. En esta práctica se implementaron las clases Empleado, Equipo y Empresa, que forman un pequeño sistema de gestión de recursos humanos: la Empresa contrata y administra a sus Empleados y a sus Equipos, mientras que un Equipo únicamente agrupa por asociación a Empleados que ya existen en la Empresa, todo orquestado a través de un menú interactivo en la clase Main.

II. Descripción de la practica

Esta práctica consistió en implementar un pequeño sistema de gestión de recursos humanos para una Empresa: los Empleados son contratados por la Empresa, pueden pertenecer a varios Equipos de proyecto a la vez, y los Equipos únicamente agrupan empleados que ya fueron contratados, sin poder crear empleados por su cuenta. El objetivo fue distinguir, directamente en el código, la relación de composición (Empresa–Empleado y Empresa–Equipo) de la relación de (Equipo–Empleado).

Lista de materiales

- Computadora personal (PC).
- Ambiente de desarrollo (editor de texto plano).
- JDK (Java Development Kit).

III. Desarrollo de la práctica

Metodología y Arquitectura

Descripción del problema y elementos clave

En este laboratorio la especificación indicaba con bastante claridad las tres entidades a modelar (Empleado, Equipo y Empresa), sus relaciones (la Empresa contrata Empleados; un Equipo agrupa Empleados que ya existen, por asociación; un mismo Empleado puede estar en varios Equipos) y los límites del sistema (máximo 50 empleados y 10 equipos por Empresa, máximo 5 miembros por Equipo). Lo que quedó como tarea propia fue decidir cómo repartir la responsabilidad entre las tres clases sin romper esas relaciones: quién crea a quién, quién valida qué, y cómo lograr que un Equipo pueda referenciar Empleados sin ser su dueño ni poder crearlos por su cuenta.

Los elementos clave del diseño son los siguientes:

- Empleado es una clase sencilla que sólo guarda su número de empleado (asignado automáticamente por la Empresa al contratarlo) y su nombre, con sus setters, getters y un toString() para poder imprimirlo dentro de los equipos.
- Equipo guarda un nombre y un arreglo de tamaño fijo de hasta 5 Empleado (sin ArrayList, por restricción de la materia), y sólo agrupa por asociación a empleados que ya le llegan como parámetro; nunca crea Empleados por su cuenta, y valida que un mismo empleado no se repita dos veces dentro del mismo equipo.
- Empresa es quien realmente crea y administra, por composición, los arreglos de Empleado y de Equipo (con sus máximos de 50 y 10 respectivamente), y es la única clase con permiso para contratar empleados y crear equipos; Main sólo orquesta estas operaciones a través de un menú interactivo, sin conocer los detalles internos de cada clase.

Datos de entrada y datos de salida

Datos de entrada

- Menú principal: número entero para elegir la operación (contratar empleado, crear equipo, asignar empleado a un equipo, mostrar empleados, mostrar equipos, cargar datos de prueba o salir).
- Contratar empleado: nombre (String) del nuevo empleado.
- Crear equipo: nombre (String) del equipo, por ejemplo "backend_poo" o "frontend_poo".
- Asignar empleado a equipo: número de empleado (int) y nombre del equipo (String) al que se quiere asignar.

Datos de salida

- Mensajes de confirmación (por ejemplo, el número de empleado asignado al contratarlo, o el nombre del equipo creado).

UABC_FCQI_PROGRAMACIÓN_ORIENTADA_A_OBJETOS.

- Mensajes de error ("Error: ...") al contratar con nombre vacío, crear un equipo repetido, asignar un empleado o equipo que no existe, o rebasar el máximo de miembros de un equipo.
- Listados impresos de los empleados y de los equipos de la empresa, incluyendo el detalle de los miembros (o "equipo vacío, sin empleados") de cada equipo.

Diagrama de clases

El siguiente diagrama muestra las clases Main, Empresa, Equipo y Empleado. Las flechas con diamante relleno representan composición: Empresa → Empleado y Empresa → Equipo, ya que es la Empresa quien crea y es dueña del ciclo de vida de sus empleados y equipos. La línea de asociación simple Equipo → Empleado indica que un Equipo sólo referencia empleados que ya existen en la Empresa, sin crearlos ni ser su dueño; por eso un mismo Empleado puede aparecer referenciado en varios Equipos al mismo tiempo.

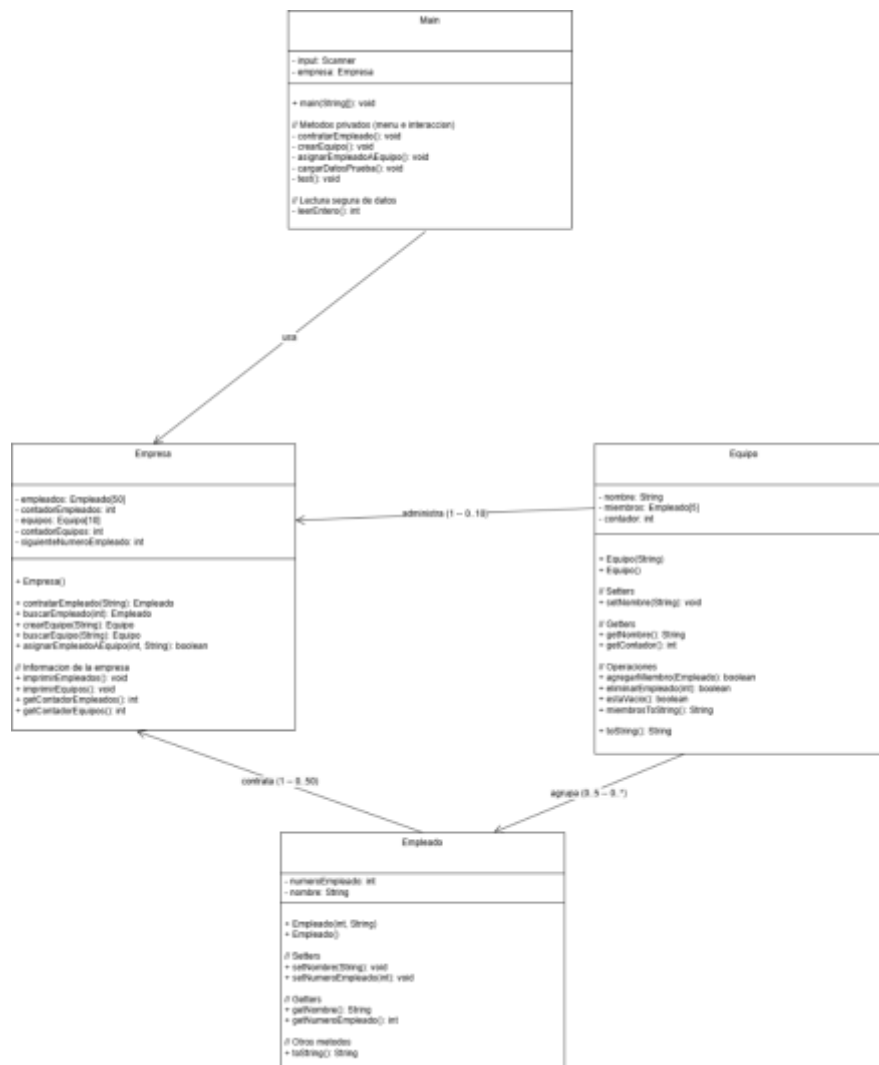


Ilustración 1: Diagrama de clases — Main, Empresa, Equipo y Empleado (composición y asociación).

Fragmentos de código y visualización de elementos clave

Clase Empresa

```
public boolean asignarEmpleadoAEquipo(int numeroEmpleado, String nombreEquipo) {
    Empleado empleado = buscarEmpleado(numeroEmpleado);
    Equipo equipo = buscarEquipo(nombreEquipo); // ambos se validan antes de continuar

    return (empleado != null && equipo != null) ? equipo.agregarMiembro(empleado) : false;
}
```

Para Empresa decidí que fuera la única clase con "permiso" para crear Empleados y Equipos: sus métodos contratarEmpleado() y crearEquipo() son los únicos que hacen new Empleado(...) y new Equipo(...), guardando el objeto en un arreglo de tamaño fijo con su propio contador y validando primero que no se haya llegado al máximo (50 empleados, 10 equipos). El método asignarEmpleadoAEquipo() fue el que más dejó clara la diferencia entre composición y asociación: primero busco al empleado y al equipo (ambos deben existir, si no se rechaza) y sólo entonces le pido al propio Equipo que agregue a ese Empleado como miembro; Empresa nunca construye un Empleado dentro de un Equipo, sólo le pasa una referencia a uno que ya contrató.

```

----- El Contratar empleados -----
Contratado: #1 - Ana Torres
Contratado: #2 - Luis Perez
Contratado: #3 - Maria Lopez

Contratar empleado con nombre vacío (inválido, debe rechazar):
Error: el nombre del empleado no puede estar vacío.

----- El los equipos solo agrupan empleados existentes -----
Asignar empleado inexistente (numero 999) a backend_poo (inválido, debe rechazar):
Error: no existe ningún empleado contratado con el numero 999.

Asignar empleado valido a un equipo inexistente (inválido, debe rechazar):
Error: no existe ningún equipo con el nombre devuena_poo.

----- El Informacion de los equipos de la empresa -----

Equipos: backend_poo
1. #1 - Ana Torres
2. #2 - Luis Perez

Equipos: frontend_poo
1. #3 - Maria Lopez
2. #1 - Ana Torres

Equipos: qa_poo
(Equipo vacío, sin empleados)

```

Ilustración 2: Ejecución de Empresa: alta de empleados y validaciones al asignar un empleado o equipo inexistente.

Clase Empleado

```
public Empleado() { this(0, ""); }

public String toString() { return String.format("#%d - %s", numeroEmpleado, nombre); }
```

Empleado quedó como una clase muy simple a propósito: sólo tiene los dos datos que la Empresa necesita para identificarlo y mostrarlo (numeroEmpleado y nombre), y ningún método que valide reglas de negocio, porque esa responsabilidad es de Empresa (quién puede contratar) y de Equipo (quién puede pertenecer a quién). Agregué un constructor vacío que delega al constructor con parámetros usando this(0, ""), y sobrescribí toString() con String.format() para poder imprimir a un empleado de forma consistente tanto en los listados de Empresa como dentro de un Equipo.

```

..... Contratar empleado .....
Nombre del empleado: Carlos Mendosa

Empleado contratado con éxito. Número de empleado: 1
Conserva este número, lo necesitas para asignarlo a un equipo.

..... Contratar empleado .....
Nombre del empleado:

Error: el nombre del empleado no puede estar vacío.

```

Ilustración 3: Ejecución de Empleado (contratarEmpleado): alta correcta y rechazo de nombre vacío.

Clase Equipo

```

public boolean agregarMiembro(Empleado empleado) {

    if (contador >= miembros.length) {
        System.out.println("Error: el equipo " + nombre + " alcanza el máximo de miembros.");
        return false;
    }
}

```

El método `agregarMiembro()` de `Equipo` fue el que más cuidé, porque ahí vive el límite de 5 miembros y la regla de que un mismo empleado no puede repetirse dentro del mismo equipo (revisada con el método privado `yaEsMiembro()`). Como el `Equipo` no conoce a la `Empresa` ni la puede consultar, decidí que simplemente recibiera el `Empleado` ya buscado como parámetro: así `Equipo` se queda enfocado únicamente en administrar su propio arreglo de miembros (agregar, eliminar, listar), y es `Empresa` la responsable de validar que ese `Empleado` exista antes de llamarlo.

```

..... 1) Crear equipos .....
Crea equipo backend_poo (0/5 miembros)
Crea equipo frontend_poo (0/5 miembros)
Crea (se deja vacío) equipo qa_poo (0/5 miembros)

Crear equipo con nombre repetido (inválido, debe rechazar):
Error: ya existe un equipo con el nombre backend_poo.

..... 2) Un empleado puede estar en múltiples equipos .....
el => frontend_poo (mismo empleado, otro equipo): true

Intentar asignar de nuevo el => backend_poo (ya es miembro, debe rechazar):
Error: el empleado Ana Torres ya pertenece al equipo backend_poo.

..... 3) Los equipos pueden estar vacíos .....
qa_poo está vacío: true

```

Ilustración 4: Ejecución de Equipo: creación de equipos, un empleado en varios equipos y un equipo vacío (qa_poo).

Clase Main — menú principal

```

public static void main(String[] args) {
    int opcion;
    do {
        // ...imprime el menú principal...
        opcion = leerEntero(); // valida que sea un entero
        switch (opcion) {
            case 1: contratarEmpleado(); break;
            case 2: crearEquipo(); break;
            case 3: asignarEmpleadoAEquipo(); break;
            case 6: cargarDatosPrueba(); break;
            case 0: System.out.println("Saliendo del programa..."); break;
            default: System.out.println("Opción no válida, intente de nuevo.");
        }
    }
}

```

```

    } while (opcion != 0);
}

```

El menú de Main quedó como un único ciclo do-while con un switch, sin submenús, porque a diferencia de otras prácticas aquí las operaciones (contratar, crear equipo, asignar, mostrar) son pocas y no tienen atributos propios que capturar en cascada. Declaré empresa como atributo estático de Main junto con el Scanner, para que todos los métodos privados compartieran la misma instancia sin pasarla como parámetro. También agregué una opción para cargar datos de prueba y una opción oculta (7, no impresa en el menú) que llama a test(), un método que ejercita automáticamente todas las reglas del enunciado (empleado en varios equipos, equipo vacío, límites, nombres repetidos, etc.) sin necesitar teclear nada, que fue mi forma principal de verificar que todo funcionara antes de probarlo a mano.

```

EMPRESA - GESTION DE RECURSOS HUMANOS
=====
1 - Contratar empleado
2 - Crear equipo de proyecto
3 - Asignar empleado a un equipo
4 - Mostrar empleados de la empresa
5 - Mostrar equipos de la empresa
6 - Cargar datos de prueba
7 - Salir del programa

Opcion: 6

Se cargaron 4 empleados y 3 equipos de prueba (backend_poe, frontend_poe y qa_poe).

Opcion: 5

----- Equipos de la empresa -----

Equipo: backend_poe
1. #1 - Ana Torres
2. #2 - Luis Perna
3. #4 - Carlos Ruiz

Equipo: frontend_poe
1. #3 - Maria Lopez
2. #4 - Carlos Ruiz

Equipo: qa_poe
(Equipo vacío, sin empleados)

Opcion: 0
Saliedo del programa...

```

Ilustración 5: Ejecución de Main: cargar datos de prueba y mostrar los equipos de la empresa desde el menú interactivo.

Reflexión y análisis crítico

Dificultades técnicas

La dificultad principal fue de diseño: decidir qué clase debía tener el "permiso" de crear objetos de las otras (composición) y cuál sólo podía referenciarlos (asociación), ya que en prácticas anteriores todas las clases se creaban más o menos libremente entre sí. Tuve que pensar dos veces el método asignarEmpleadoAEquipo() para que viviera en Empresa y no en Equipo, precisamente para que Equipo nunca tuviera que buscar directamente en la lista de empleados de la Empresa.

Fracasos, fallas, bugs y errores

Al principio había puesto la validación de "empleado ya es miembro del equipo" (yaEsMiembro()) después de insertarlo en el arreglo en lugar de antes, lo que provocaba que un mismo empleado apareciera duplicado dentro del mismo equipo si se le asignaba dos veces seguidas. También olvidé dejar en null el hueco al eliminar un miembro del arreglo, lo que hubiera dejado un miembro duplicado al final del arreglo.

Cómo se resolvió

Corregí el orden de las validaciones en `agregarMiembro()` para que `yaEsMiembro()` se revisara antes de insertar, y usé el método `test()` de `Main` para confirmarlo: ahí quedó explícito el caso "Intentar asignar de nuevo e1 -> backend_poo (ya es miembro, debe rechazar)". Para `eliminarEmpleado()` recorrí el arreglo recorriendo los índices posteriores al elemento eliminado y puse `contador--` al final para dejar limpia la última posición.

Conclusión

Esta práctica me ayudó a entender la diferencia entre asociación y composición no sólo de forma teórica, sino viéndola reflejada directamente en el código: quién tiene el `new` de quién, y quién sólo recibe una referencia ya existente. Tener que decidir en qué clase vivía cada validación (Empresa vs. Equipo) me hizo pensar más en el diseño antes de escribir código, en lugar de resolver todo dentro de una sola clase gigante.

IV. Bibliografía

- [1] Oracle Corporation. (2024). Java SE Documentation. Recuperado de <https://docs.oracle.com/javase/>
- [2] Deitel, P. J., & Deitel, H. M. (2016). Java: How to Program (10th ed.). Pearson Education.